



MMM 5162 – Modelling and Simulation

Mini Project 1

Computational Simulation and Parameter Study

[CLO 3.1, PLO V2]

Two-Link Robot Pick-Cycle Kinematic Simulation

Track MR, Student-Specific Case $S = 18$

Submitted To

Prof. Adel Abdennour

Student's Name

Mohammed Rasheed

Student ID: 481011696

Date

13 September 2026

1. Introduction

This project simulates a two-link planar robot performing one smooth pick-and-place transfer. The supplied model was implemented as a reusable Python function, and the assigned case $S = 18$ was evaluated using the nominal cycle time together with $0.8T$ and $1.2T$. The objective is to quantify how cycle duration affects joint-speed requirements while checking whether the geometric end-effector path changes. The work follows the simulation workflow and reporting practices introduced in Topic 1 [1, 2].

2. Theory/Modelling

For $S = 18$, the assigned parameters are $L_1 = 0.470$ m, $L_2 = 0.372$ m, $q_{1,start} = 38^\circ$, $q_{1,end} = 66^\circ$, $q_{2,start} = -40.6^\circ$, $q_{2,end} = 24^\circ$, and $T = 3.12$ s. Angles are converted to radians internally as required by the assignment [2]. The prescribed motion is

$$u = \frac{t}{T}, \quad s(u) = 3u^2 - 2u^3,$$

$$q_j(t) = q_{j,start} + (q_{j,end} - q_{j,start}) s(u), \quad j = 1, 2,$$

with angular velocity

$$\dot{q}_j(t) = (q_{j,end} - q_{j,start}) \frac{6u - 6u^2}{T}.$$

The end-effector coordinates follow from the two-link forward kinematics

$$x = L_1 \cos q_1 + L_2 \cos(q_1 + q_2), \quad y = L_1 \sin q_1 + L_2 \sin(q_1 + q_2).$$

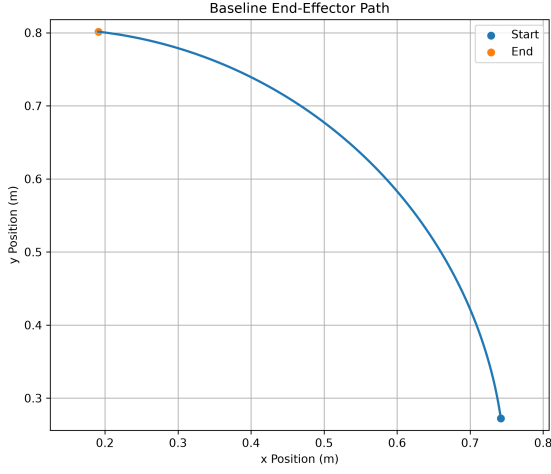
Because $6u - 6u^2 = 0$ at $u = 0$ and $u = 1$, the commanded joint velocities are zero at both ends of the cycle.

3. Simulation and Design Methods

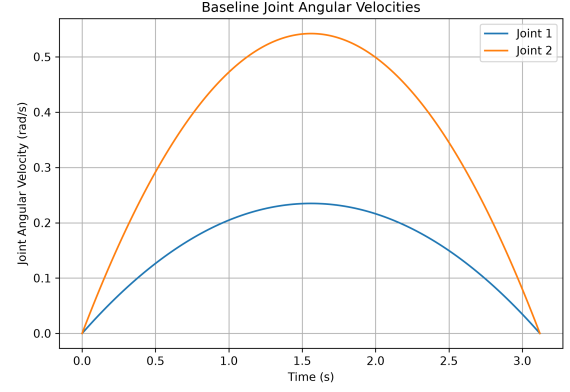
Python 3.13 was used with NumPy, pandas, and Matplotlib. The kinematic equations were placed in `robot_model.py`, while `main_simulation.py` generated 1001 time samples, executed the parameter study, calculated performance measures, produced figures, and saved the numerical results. Three cycle times were evaluated: 2.496 s, 3.120 s, and 3.744 s.

Automatic checks reject invalid link lengths, non-positive cycle time, and time values outside $0 \leq t \leq T$. The program also verifies the assigned initial and final angles and confirms zero initial and final joint velocities.

Figure 1 presents the baseline trajectory and joint velocities.



(a) Baseline end-effector path.



(b) Baseline joint angular velocities.

Figure 1: Principal responses for the nominal cycle $T = 3.12$ s.

4. Discussion/Conclusions

Table 1 summarizes the parameter-study results.

Table 1: Quantitative performance measures.

Case	T (s)	Max. $ \dot{q}_1 $ (rad/s)	Max. $ \dot{q}_2 $ (rad/s)	Path (m)
$0.8T$	2.496	0.29369	0.67757	0.82045
T	3.120	0.23495	0.54206	0.82045
$1.2T$	3.744	0.19579	0.45172	0.82045

Reducing the cycle time from T to $0.8T$ increases both maximum joint speeds by 25%, whereas increasing it to $1.2T$ reduces them by approximately 16.7%. Maximum end-effector speed follows the same trend: 0.49836 m/s, 0.39869 m/s, and 0.33224 m/s for $0.8T$, T , and $1.2T$, respectively.

The path length remains 0.82045 m in all cases. The end-effector moves from approximately (0.74198, 0.27249) m to (0.19117, 0.80137) m. Thus, changing T changes the speed at which the same normalized sequence of configurations is traversed but does not change the prescribed geometric path.

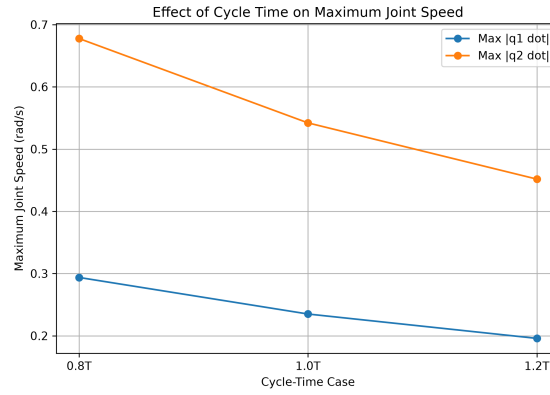


Figure 2: Effect of cycle time on maximum joint angular speed.

The principal engineering conclusion is that a faster execution of the same pick-and-place path requires greater actuator-speed capability. Therefore, reducing cycle time improves throughput but increases the joint-speed requirement.

Reproducibility and responsible use: All figures and numerical values were generated directly from the submitted $S = 18$ Python simulation. The model assumes rigid planar links and ideal prescribed joint motion; dynamics, torque limits, flexibility, and external disturbances are outside its scope. Generative AI supported explanation, coding guidance, debugging, and documentation, while execution, checking, interpretation, and final responsibility remained with the student.

References

- [1] A. Abdenmour, "MATLAB and Python Tools for Engineering Simulation," *MMM 5162 Modelling and Simulation*, Topic 1 lecture notes, Islamic University of Madinah, 2026.
- [2] A. Abdenmour, "Mini Project 1: Computational Simulation and Parameter Study," *MMM 5162 Modelling and Simulation*, Islamic University of Madinah, Fall Semester 2026.

A. Source Code

The complete executable Python source code used to generate the simulation results, figures, and numerical tables is available in the public GitHub repository below.

GitHub Repository: MMM 5162 Mini Project 1

<https://github.com/m20m03m-prog/Modulation-Mini-Project-1>

The repository contains the reusable robot model, the main simulation program, generated outputs, and reproducibility instructions.